

Individual formative Assignment

Module : Introduction to Blockchain Development

Date : 21st September

Repository:

https://github.com/josep-prog/formative_introduction_blockchain.git

File Name	Link	Purpose of the file
Blockchain.c	blockchain.c	I created this file to control the whole program. It handles the menu and calls the other files when the program needs to load records, create blockchain transactions, or perform security checks.
Blockchain.h	blockchain.h	I created this file to manage the blockchain. It creates borrowing and returning blocks, links them using hashes, checks active loans, and validates the blockchain.
Crypto.c	crypto.c	I created this file to define the Block structure and declare the blockchain functions so that other files, especially main.c, can use them.

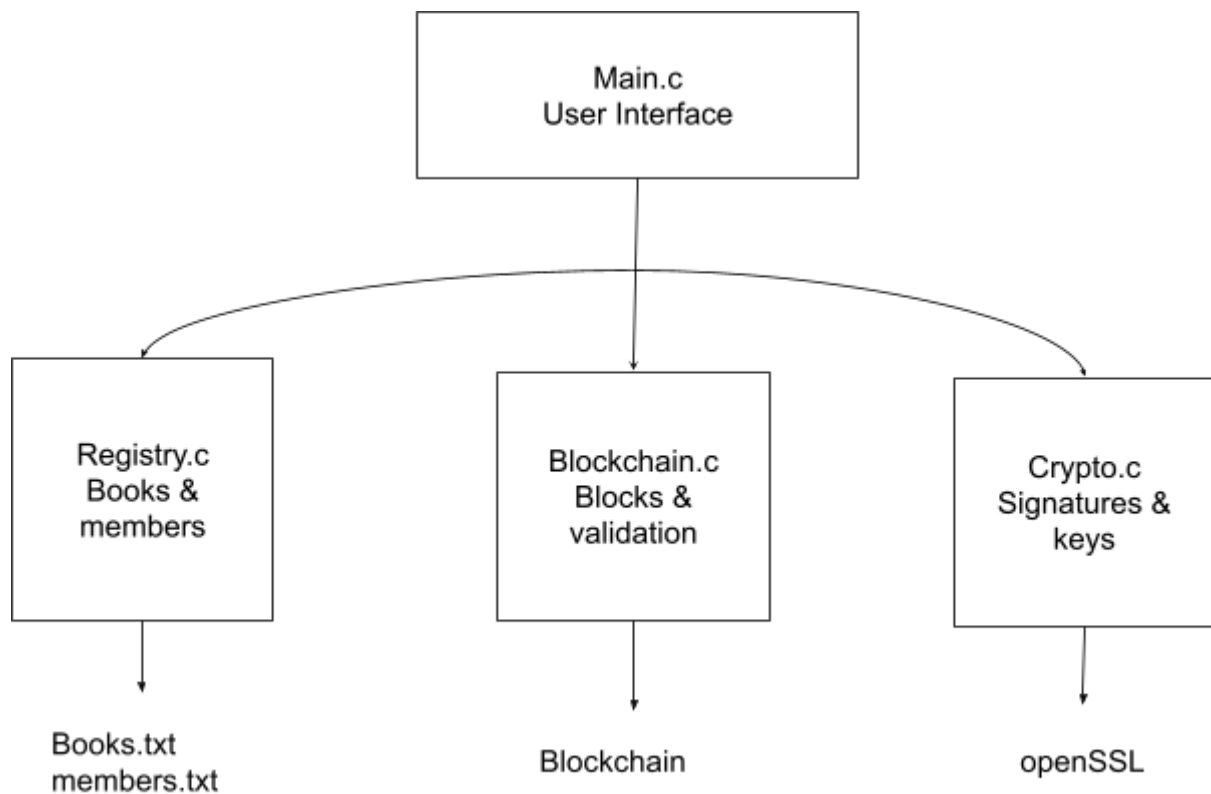
Crypto.h	crypto.h	I created this file to handle the security part of the system. It generates keys, creates digital signatures, and verifies signatures using OpenSSL.
Main.c	main.c	I created this file to declare the cryptographic functions so that main.c and blockchain.c can use the security functions from crypto.c
Registry.h	registry.h	I created this file to manage books and members. It loads their information from the data files and provides functions to find a specific book or member.
Registry.c	registry.c	I created this file to define the Book and Member structures and declare the registry functions used by main.c.
Books.txt	books.txt	This file is for storing the registered books, including their IDs, titles, and authors.
Members.txt	members.txt	this file to store the registered library members and their

		basic information.
Makefile	<u>Makefile</u>	this file to make compiling the whole project easier by providing the commands needed to build the program and link OpenSSL.

How My Blockchain-Based Library Lending Tracker Works

A traditional library lending system can be as simple as keeping a list of which books have been borrowed and returned. However, such a record can be changed after the transaction has happened. For example, someone could change a record from “borrowed” to “returned” without leaving an obvious sign that the record was changed. The purpose of my project was to create a small library lending system where such changes could be detected. To achieve this, I used a simple blockchain written in C. Instead of keeping each lending action as an independent record, every borrow and return is stored as a block. The blocks are connected to each other using their digital fingerprints, called hashes. If an older record is changed, its fingerprint also changes, which allows the program to detect that the chain has been altered. This approach follows the main purpose of the assignment, which requires a blockchain structure, SHA-256 hashing, digital signatures, validation, and a simple interface for managing lending records.

Approach for Entire project



I divided the program into different parts so that each part has a clear responsibility. The main.c file controls what the user sees and connects the other parts of the program. The registry.c file is responsible for loading and searching for books and members. The blockchain.c file contains the main blockchain logic, such as creating blocks, connecting them, checking borrowing status, calculating hashes, and validating the chain. The crypto.c file handles the digital signatures and the creation of the cryptographic key. This separation makes the program easier to understand because the code that deals with books and members is kept separate from the code that deals with the blockchain and cryptography.

Run the project

1. Clone : git clone

https://github.com/josep-prog/formative_introduction_blockchain.git

2. cd formative_introduction_blockchain

Command	Tests
sudo apt install gcc make libssl-dev	Is for installing the C compiler , build tool and openssl development files and the program needs
make clean && make	Deletes the old program file and compiles all the source files again into the new program called library
./library	To start the program and it shows the menu

Books :

ID	Title	Author
BK001	The Money Trap: Lost Illusions Inside the Tech Bubble	Alok Sama
BK002	Wars Guns & Votes: Democracy in Dangerous Places	Paul Collier
BK003	Sustainable Leadership	Daniel W. Graham
BK005	Gulliver's Travels and Other Writings	Jonathan Swift

Member:

ID	Name	
ALU001	Irakoze Jean	BSE
ALU002	Joseph Habimana	BSE
ALU003	Uwase Diane	BEL
ALU004	Manzi Eric	BSE
ALU005	Mukamana Alice	BEL

What to expect :

1.

```
joe@code:~/library-blockchain/formative_introduction_blockchain
joe@code:~/library-blockchain/formative_introduction_blockchain$ udo apt install gcc make libssl-dev
Command 'udo' not found, but can be installed with:
sudo apt install udo
joe@code:~/library-blockchain/formative_introduction_blockchain$ sudo apt install gcc make libssl-dev
[sudo] password for joe:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
gcc is already the newest version (4:13.2.0-7ubuntu1).
make is already the newest version (4.3-4.1build2).
libssl-dev is already the newest version (3.0.13-0ubuntu3.15).
libssl-dev set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 50 not upgraded.
joe@code:~/library-blockchain/formative_introduction_blockchain$ make clean && make
rm -f library
gcc -Wall -Wextra -std=c11 -o library src/main.c src/blockchain.c src/crypto.c src/registry.c -lssl -lcrypto
joe@code:~/library-blockchain/formative_introduction_blockchain$ gcc -Wall -Wextra -std=c11 -o library src/main.c src/blockchain.c src/crypto.c src/registry.c
-lssl -lcrypto
joe@code:~/library-blockchain/formative_introduction_blockchain$ ls
data  library  Makefile  README.md  src
joe@code:~/library-blockchain/formative_introduction_blockchain$
```

2.

```
joe@code:~/library-blockchain/formative_introduction_blockchain$ ls
data library Makefile README.md src
joe@code:~/library-blockchain/formative_introduction_blockchain$ ./library
Loaded 5 books and 5 members.
Digital signing key generated.

LIBRARY MENU
1. Borrow a book
2. Return a book
3. View all lending records
4. Validate the blockchain
5. Tamper-detection demo
6. Exit
Choice: 1
Book ID : BK001
Member ID: ALU001
Borrowed 'The Money Trap: Lost Illusions Inside the Tech Bubble' for Irakoze Jean.

LIBRARY MENU
1. Borrow a book
2. Return a book
3. View all lending records
4. Validate the blockchain
5. Tamper-detection demo
6. Exit
Choice: 2
Book ID: BK001
Returned 'The Money Trap: Lost Illusions Inside the Tech Bubble'.

LIBRARY MENU
1. Borrow a book
2. Return a book
3. View all lending records
4. Validate the blockchain
5. Tamper-detection demo
6. Exit
Choice: 3
-----
Block #0 [GENESIS]
Time : Mon Sep 21 12:38:22 2026
Hash : 4fa96643a29f474235ee7d2b46f28510ff4f1772ee66206578763b5a0f176c51
Prev : 0000000000000000000000000000000000000000000000000000000000000000
Signature: n/a (genesis block)
```

3.

```
joe@code:~/library-blockchain/formative_introduction_blockchain$ ./library
-----
Block #0 [GENESIS]
Time : Mon Sep 21 12:38:22 2026
Hash : 4fa96643a29f474235ee7d2b46f28510ff4f1772ee66206578763b5a0f176c51
Prev : 0000000000000000000000000000000000000000000000000000000000000000
Signature: n/a (genesis block)
-----
Block #1 [BORROWED]
Book : The Money Trap: Lost Illusions Inside the Tech Bubble (BK001)
Member : Irakoze Jean (ALU001)
Time : Mon Sep 21 12:38:54 2026
Hash : 5997c83299df76c7ac583548301cdb0e9d22ccf594938dfb4afe8bd56a9c781f
Prev : 4fa96643a29f474235ee7d2b46f28510ff4f1772ee66206578763b5a0f176c51
Signature: VALID (71 bytes)
-----
Block #2 [RETURNED]
Book : The Money Trap: Lost Illusions Inside the Tech Bubble (BK001)
Member : Irakoze Jean (ALU001)
Time : Mon Sep 21 12:39:08 2026
Hash : f486e3ae71abb5ca2b17bffc8d445d80809fb1b61253bca1d7fc3485d1745394
Prev : 5997c83299df76c7ac583548301cdb0e9d22ccf594938dfb4afe8bd56a9c781f
Signature: VALID (69 bytes)
-----
LIBRARY MENU
1. Borrow a book
2. Return a book
3. View all lending records
4. Validate the blockchain
5. Tamper-detection demo
6. Exit
Choice: 4
Blockchain is VALID - no tampering detected.

LIBRARY MENU
1. Borrow a book
2. Return a book
3. View all lending records
4. Validate the blockchain
5. Tamper-detection demo
6. Exit
Choice: 5
```

4.

```
-----
Block #2 [RETURNED]
Book : The Money Trap: Lost Illusions Inside the Tech Bubble (BK001)
Member : Irakoze Jean (ALU001)
Time : Mon Sep 21 12:39:08 2026
Hash : f486e3ae71abb5ca2b17bffc8d445d80809fb1b61253bca1d7fc3485d1745394
Prev : 5997c83299df76c7ac583548301cdb0e9d22ccf594938dfb4afe8bd56a9c781f
Signature: VALID (69 bytes)

LIBRARY MENU
1. Borrow a book
2. Return a book
3. View all lending records
4. Validate the blockchain
5. Tamper-detection demo
6. Exit
Choice: 4
Blockchain is VALID - no tampering detected.

LIBRARY MENU
1. Borrow a book
2. Return a book
3. View all lending records
4. Validate the blockchain
5. Tamper-detection demo
6. Exit
Choice: 5
This will edit Block #1's stored book title in memory,
WITHOUT re-signing or re-hashing it - exactly what an
attacker trying to rewrite history would do.
Block #1's book title has been altered.
Running chain validation...
Blockchain is INVALID - tampering detected, as expected.
(Restart the program to get a clean chain again.)

LIBRARY MENU
1. Borrow a book
2. Return a book
3. View all lending records
4. Validate the blockchain
5. Tamper-detection demo
6. Exit
Choice:
```

Books and Members

Before the system can record any lending activity, it first loads the book and member information from two files: books.txt and members.txt. The assignment requires these files to be loaded when the program starts and used to check whether the supplied book and member IDs are valid.

The book file contains the book ID, title, and author, while the member file contains the member ID, full name, and course code. The program stores these records in simple arrays while it is running. The functions `load_books()` and `load_members()` are responsible for reading the two files. If either file is missing, empty, or cannot provide valid records, the program reports an error and stops. This is important because the system should not record a lending transaction if it cannot confirm that the book or member exists.

When a user wants to borrow a book, the program uses `find_book()` to search for the book ID and `find_member()` to search for the member ID. If either one cannot be found, the transaction is rejected. The comparison is also exact, meaning that an ID such as BK001EXTRA cannot be treated as

the same as BK001. In this way, the registry acts as the first level of protection before anything is added to the blockchain.

Testing :

<i>command</i>	<i>purpose</i>
printf '1\nBK001\nALU001\n6\n' ./library	Borrows book BK001 ALU001 and the exits , which checks that a normal borrow works.
printf '1\nBK001\nALU001\n1\nBK001\nALU002\n6\n' ./library	Borrows BK001 for one member and then tries to borrow the same for another member, which checks that the program refuses a book that is already on loan.
printf '1\nBK999\nALU001\n6\n' ./library	Tries to borrow a book that is not in the registry, which checks that program print rejects unknown members too.
printf '1\nBK001\nALU001\n2\nBK001\n6\n' ./library	Borrows a book and then returns it , which checks that a return block is created.
printf '2\nBK001\n6\n' ./library	Tries to return a book that was never borrowed , which checks that the program prints an error.
printf '1\nBK001\nALU001\n2\nBK001\n3\n6\n' ./library	Borrows and returns a book and then shows all records , which checks that every block is printed with a VALID signature
printf '1\nBK001\nALU001\n4\n6\n' ./library	Borrows a book, changes a past block, and validates twice, which checks that tampering is detected and stays detected

<code>printf 'abc\n6\n' ./library</code>	Types letters where a number is expected, which checks that the program ignores bad menu input and does not crash.
<code>printf " ./library</code>	Sends no input at all, which checks that the program exists cleanly and does not hang when the input closes.

What to expect:

1.

```
joe@code:~/library-blockchain/formative_introduction_blockchain$ printf '1\nBK001\nALU001\n6\n' | ./library
Loaded 5 books and 5 members.
Digital signing key generated.

LIBRARY MENU
1. Borrow a book
2. Return a book
3. View all lending records
4. Validate the blockchain
5. Tamper-detection demo
6. Exit
Choice: Book ID : Member ID: Borrowed 'The Money Trap: Lost Illusions Inside the Tech Bubble' for Irakoze Jean.

LIBRARY MENU
1. Borrow a book
2. Return a book
3. View all lending records
4. Validate the blockchain
5. Tamper-detection demo
6. Exit
Choice: Goodbye.
joe@code:~/library-blockchain/formative_introduction_blockchain$ printf '1\nBK001\nALU001\n1\nBK001\nALU002\n6\n' | ./library
Loaded 5 books and 5 members.
Digital signing key generated.

LIBRARY MENU
1. Borrow a book
2. Return a book
3. View all lending records
4. Validate the blockchain
5. Tamper-detection demo
6. Exit
Choice: Book ID : Member ID: Borrowed 'The Money Trap: Lost Illusions Inside the Tech Bubble' for Irakoze Jean.

LIBRARY MENU
1. Borrow a book
2. Return a book
3. View all lending records
4. Validate the blockchain
5. Tamper-detection demo
6. Exit
Choice: Book ID : Member ID: ERROR: This book is already on loan.

LIBRARY MENU
```

2.

```
joe@code:~/library-blockchain/formative_introduction_blockchain
Loaded 5 books and 5 members.
Digital signing key generated.

LIBRARY MENU
1. Borrow a book
2. Return a book
3. View all lending records
4. Validate the blockchain
5. Tamper-detection demo
6. Exit
Choice: Book ID : Member ID: ERROR: Book or Member not found

LIBRARY MENU
1. Borrow a book
2. Return a book
3. View all lending records
4. Validate the blockchain
5. Tamper-detection demo
6. Exit
Choice: Goodbye.
joe@code:~/library-blockchain/formative_introduction_blockchain$ printf '1\nBK001\nALU001\n2\nBK001\n6\n' | ./library
Loaded 5 books and 5 members.
Digital signing key generated.

LIBRARY MENU
1. Borrow a book
2. Return a book
3. View all lending records
4. Validate the blockchain
5. Tamper-detection demo
6. Exit
Choice: Book ID : Member ID: Borrowed 'The Money Trap: Lost Illusions Inside the Tech Bubble' for Irakoze Jean.

LIBRARY MENU
1. Borrow a book
2. Return a book
3. View all lending records
4. Validate the blockchain
5. Tamper-detection demo
6. Exit
Choice: Book ID: Returned 'The Money Trap: Lost Illusions Inside the Tech Bubble'.
```

3.

```
joe@code:~/library-blockchain/formative_introduction_blockchain
Loaded 5 books and 5 members.
Digital signing key generated.

LIBRARY MENU
1. Borrow a book
2. Return a book
3. View all lending records
4. Validate the blockchain
5. Tamper-detection demo
6. Exit
Choice: Book ID: ERROR: This book is not currently on loan.

LIBRARY MENU
1. Borrow a book
2. Return a book
3. View all lending records
4. Validate the blockchain
5. Tamper-detection demo
6. Exit
```

The Blockchain and the Block

The main idea of the system is that every important lending event becomes a block. A block contains information such as the book, the member, the action taken, the time of the action, the previous block's hash, a digital signature, and its own hash. The assignment requires the block to contain these important pieces of information so that each lending event can be properly recorded.

For example, when a member borrows a book, the program creates a block containing the book ID and title, the member ID and name, and the action BORROWED. When the book is returned, another block is created with the action RETURNED. I also copy the book title and member name into the block at the time of the transaction. This means that the block keeps the information that was recorded at that particular moment instead of depending on the registry later.

The first block is called the **genesis block**. It does not represent a real borrowing or returning action. Its purpose is simply to start the blockchain. Its previous hash is set to 64 zeros because there is no block before it. Every other block then stores the hash of the block immediately before it. The assignment specifically requires this genesis block and the connection between blocks through their previous hashes.

This creates a chain such as:

Genesis → Borrow → Return → Borrow → Return

The important point is that a block does not stand alone. It remembers the previous block, which makes changes to earlier records easier to detect.

Hashing and Detecting Changes

A hash can be understood as a digital fingerprint of information. In my program, SHA-256 is used to create this fingerprint. Before calculating the hash, the program puts the important information from the block into one consistent piece of data. The information includes the block number, time, book, member, action, and previous hash. The signature information is also included when the final block hash is calculated.

The function `create_transaction_data()` was created for this purpose. Its job is simply to prepare the important information from a block in one consistent format. This is useful because the same information needs to be used when creating, signing, and checking a transaction.

The `calculate_hash()` function then takes this information and creates the block's SHA-256 fingerprint. When a block is created, the fingerprint is

stored inside the block. Later, when the blockchain is checked, the program calculates the fingerprint again and compares it with the stored one. If the two are different, the contents of the block have changed.

For example, suppose a block originally contains the title “Things Fall Apart.” If someone changes it to “TAMPERED TITLE,” the block's fingerprint will no longer be the same. This gives the system a simple way to detect that the record has been changed.

Digital Signatures

In addition to hashing, the program uses digital signatures for lending transactions. A digital signature can be thought of as a special digital stamp attached to a transaction. When the program starts, it creates a pair of cryptographic keys. The private key is used to sign borrow and return transactions, while the other part of the key pair is used to check the signature later.

The function `generate_key_pair()` creates this key pair. The function `sign_data()` creates a signature for a lending transaction, and `verify_signature()` checks whether the signature is still valid. When the user chooses to view the records, the program checks the signatures and reports whether they are valid or invalid.

The purpose of using signatures here is to provide another way of checking the lending records. Hashing helps the program detect changes in the data, while the signature provides a way to check the authenticity of the transaction. The assignment specifically requires digital signatures to authenticate lending actions.

Borrowing a Book

The borrowing process follows a simple sequence. First, the user enters a book ID and a member ID. The program checks both IDs against the registries. If either ID is unknown, the program stops the transaction and displays an error.

If both IDs are valid, the program then checks whether the book is already on loan. For this, I created the `find_active_borrow()` function. Instead of keeping a separate list of borrowed books, this function looks through the blockchain from the newest record backwards. It finds the latest transaction involving that book.

If the latest transaction says `BORROWED`, the book is currently out. If the latest transaction says `RETURNED`, the book is available. This means that the blockchain itself is used to determine the current state of the book.

If the book is available, the program calls `create_borrow_block()`. This creates a new block, connects it to the previous block, signs the transaction, calculates its hash, and adds it to the end of the blockchain. This follows the assignment's required process of checking the IDs and the book's loan status before creating a new borrowing record.

Returning a Book

The return process is similar but does not require the member to enter their ID again. The user only provides the book ID. The program first confirms that the book exists in the registry. It then uses `find_active_borrow()` to look for the book's most recent borrowing record.

If there is no active borrowing record, the program does not create a return block. This prevents someone from returning a book that the system does not consider borrowed.

If the book is currently borrowed, the program creates a new `RETURNED` block. It takes the member information from the original borrowing block, so the return record still shows who originally borrowed the book. The new block is then signed, given its hash, and added to the chain. This follows the assignment requirement for handling returns.

Validating the Blockchain

One of the most important functions in the program is `validate_chain()`. Its purpose is to answer a simple question: **Has anything in the blockchain been changed?**

The function checks every block in two ways. First, it calculates the block's hash again and compares it with the hash already stored in that block. If they are different, the contents of the block have changed. Second, for every block after the first one, it checks whether the block's `previous_hash` still matches the actual hash of the block before it.

Both checks are important. The first check tells us whether the contents of a block have changed. The second check makes sure that the blocks are still correctly connected. Together, they provide the basic tamper-detection mechanism required by the assignment.

While developing the program, I also found an issue with validation. Recalculating the hash directly on the original block could change the stored hash during the checking process. I solved this by making a copy of the block before recalculating its hash. In simple terms, the program now checks a copy instead of changing the original record. This means that validation only checks the blockchain and does not modify it.

Tamper Detection Demonstration

The program includes a specific option to demonstrate what happens when an old record is changed. After at least one book has been borrowed, the program changes the title stored in Block #1 to TAMPERED TITLE. It does not update the hash or create a new signature.

The program then validates the blockchain. Because the contents of the block no longer match its stored fingerprint, the validation fails and the program reports that tampering has been detected. This provides a simple demonstration of the main reason for using a blockchain in this project. The assignment also requires a demonstration that changing a past block breaks chain validation.

The change is only made in memory. Therefore, restarting the program creates a new clean blockchain for another demonstration.

Error Handling

The program also tries to handle situations where an operation should not be allowed. It checks whether the registry files exist, whether they contain usable records, whether book and member IDs are valid, whether a book is already borrowed, and whether a book is actually on loan before allowing it to be returned. It also prevents new blocks from being added once the blockchain reaches its maximum size.

The program also handles invalid menu input. If the user enters something that is not a number when a menu choice is expected, the program clears the invalid input and shows the menu again. If the input stream is closed, the program exits instead of continuing to ask for input. These checks help prevent the program from entering an unexpected state.

Current Limitations

Although the system demonstrates the main ideas required for the blockchain part of the assignment, it also has some limitations. The blockchain is currently stored only in memory, so the borrow and return records disappear when the program closes. The book and member registries are loaded from files, but the blockchain itself is not yet saved to a file. Therefore, the data persistence requirement is only partially implemented. The assignment lists file-based data persistence as one of its objectives.

The program also does not currently have a login or role system. This means that anyone who can access the menu can perform the available operations. The signing key is also generated when the program starts and is not permanently stored. In addition, the blockchain validation function currently checks the hashes and the links between blocks, while signature checking is performed separately when records are viewed.

Finally, the block structure allows an OVERDUE action, as described in the assignment, but the current program only creates BORROWED and RETURNED blocks. These are limitations of the current implementation rather than hidden features of the system.

The main purpose of this project was to apply the basic idea of blockchain to a library lending problem. Instead of treating a lending record as an ordinary piece of information that can simply be changed, the program creates a history of connected records. Each new record is linked to the previous one through its hash, and lending actions are also digitally signed.

The process is therefore straightforward. The program first checks that the book and member exist. It then checks the book's previous lending activity. If the operation is valid, it creates a new block, signs it, calculates its hash, and adds it to the chain. Later, the program can examine the chain and determine whether any of its records have been changed.

The project therefore demonstrates the basic relationship between a library system and blockchain: **the registry tells the system what is valid, the blockchain keeps the history, cryptography helps protect the records, and validation checks whether the history has been altered.**